

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE TRABAJO PRÁCTICO

INFORMACIÓN BÁSICA					
ASIGNATURA:	LABORATORIO PROGRAMACIÓN WEB 2				
TÍTULO DEL TRABAJO:	LAB 02				
NÚMERO DE TRABAJO:	02	AÑO LECTIVO:	2023	NRO. SEMESTRE:	1°
FECHA DE PRESENTACIÓN	22/05/23	HORA DE PRESENTACIÓN	hh/mm/ss		
INTEGRANTE (s)	Sebastian Arley Chirinos Negrón			NOTA (0-20)	
DOCENTE(s):	RICHART SMITH ESCOBEDO QUISPE				

INTRODUCCIÓN
OBJETIVOS TEMAS Y COMPETENCIAS OBJETIVOS Realizar ejercicios en temas de Estructuras de datos, tipos de datos abstractos, bucles, Arrays, Listas enlazadas, Recursión. TEMAS TAD Arrays COMPETENCIAS C.m. Construye responsablemente soluciones haciendo uso de estructuras de datos y algoritmos, siguiendo un proceso adecuado para resolver problemas computacionales que se ajustan al uso de los recursos disponibles y a especificaciones concretas. CONTENIDO DE LA GUÍA MARCO CONCEPTUAL https://www.w3schools.com/java/ https://docs.oracle.com/javase/7/docs/api/java/util/List.html https://docs.oracle.com/javase/tutorial/java/generics/types.html

EJERCICIOS

cabe recalcar que en cada uno de los ejercicios puse comentarios para poder entender mucho mejor el código

Ejercicio 1:

```
1 public class Ejercicio01 {
2     public static void main(String args[]) {
3         int [] A = {1,2,3}; // Creamos un array A con elementos 1, 2 y 3
4         int [] B = invertirArray(A); // Llamamos al método invertirArray y pasamos el array A como argumento
5         imprimir(B); // Llamamos al método imprimir y pasamos el array invertido B como argumento
6     }
7
8     // Método para invertir un array
9     1 usage
10    @ public static int[] invertirArray(int[] A){
11        int [] Ain = new int[A.length]; // Creamos un nuevo array Ain con la misma longitud que el array A
12        for (int i = 0, j = A.length - 1; i < Ain.length; i++, j--) {
13            Ain[j] = A[i]; // Asignamos el elemento en la posición i del array A al elemento en la posición j del array Ain
14        }
15        return Ain; // Devolvemos el array invertido Ain
16    }
17
18    // Método para imprimir un array
19    1 usage
20    @ public static void imprimir(int [] a) {
21        for (int i = 0; i < a.length; i++) {
22            System.out.print(a[i] + " "); // Imprimimos cada elemento del array separada por un espacio
23        }
24        System.out.println(); // Imprimimos una nueva línea después de imprimir el array completo
25    }
26 }
```

Ejercicio 2:

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

```

1 ▶ public class Ejercicio02 {
2 ▶     public static void main(String[] args) {
3         int [] A = {1,2,3,4,5}; // Creamos un array A con elementos 1, 2, 3, 4 y 5
4         int [] B = rotarIzquierdaArray(A, 2); // Llamamos al método rotarIzquierdaArray y pasamos el array A y el valor 2 como argumentos
5         imprimir(B); // Llamamos al método imprimir y pasamos el array rotado B como argumento
6     }
7
8     // Método para rotar un array hacia la izquierda
9     1 usage
10    @ public static int[] rotarIzquierdaArray(int[] A, int d){
11        int[] Aiz = new int[A.length]; // Creamos un nuevo array Aiz con la misma longitud que el array A
12        for (int i = 0; i < A.length; i++) {
13            int j = (i + (A.length - d)) % A.length; // Calculamos la nueva posición j para el elemento en la posición i después de la rotación
14            Aiz[j] = A[i]; // Asignamos el elemento en la posición i del array A al elemento en la posición j del array Aiz
15        }
16        return Aiz; // Devolvemos el array rotado Aiz
17    }
18
19    // Método para imprimir un array
20    1 usage
21    @ public static void imprimir(int [] a) {
22        for (int i = 0; i < a.length; i++) {
23            System.out.print(a[i] + " "); // Imprimimos cada elemento del array separado por un espacio
24        }
25        System.out.println(); // Imprimimos una nueva línea después de imprimir el array completo
26    }
27 }

```

Ejercicio 3:

```

▶ public class Ejercicio03 {
2 usages
    static int contador = 1; // Variable estática para contar el número de asteriscos en cada nivel del triángulo
▶
    public static void main(String[] args) {
        trianguloRecursivo( base: 4); // Llamamos al método trianguloRecursivo con el valor de base igual a 4
    }

2 usages
    public static void trianguloRecursivo(int base){
        // Si la base llega a ser 0, no hay nada más que imprimir y se termina la recursión
        if(base == 0) {
            return;
        } else {
            // Imprimimos asteriscos en cada nivel del triángulo
            for (int i = 0; i < contador; i++) {
                System.out.print("*");
            }
            System.out.println(); // Imprimimos una nueva línea después de imprimir los asteriscos del nivel actual

            contador++; // Incrementamos el contador para el siguiente nivel del triángulo
            trianguloRecursivo( base: base - 1); // Llamamos de forma recursiva al método, disminuyendo la base en 1 en cada llamada
        }
    }
}

```

CUESTIONARIO

¿Qué diferencia hay entre un List y un ArrayList en Java?

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

List: List es una interfaz que define las operaciones básicas que se pueden realizar en una lista, como agregar, eliminar, acceder a elementos, entre otros. Es una interfaz de nivel superior y no se puede instanciar directamente. Al ser una interfaz, puede ser implementada por diferentes clases, incluyendo ArrayList.

ArrayList: ArrayList es una clase concreta que implementa la interfaz List. Proporciona una implementación basada en un array dinámico, lo que significa que el tamaño del ArrayList puede crecer dinámicamente según sea necesario. Esto permite agregar y eliminar elementos de manera eficiente, así como acceder rápidamente a los elementos por su índice. Sin embargo, esta eficiencia puede verse afectada si se realizan operaciones frecuentes de inserción o eliminación en posiciones intermedias de la lista.

¿Qué beneficios y oportunidades ofrecen las clases genéricas en Java?

Reutilización de código: Las clases genéricas permiten escribir código que pueda trabajar con diferentes tipos de datos, lo que promueve la reutilización del código en diferentes contextos y evita la necesidad de duplicar código para tratar tipos diferentes.

Seguridad en tiempo de compilación: El uso de clases genéricas proporciona seguridad en tiempo de compilación al especificar el tipo de datos que se utilizará en la clase. Esto permite detectar y corregir errores de tipo en el momento de compilación, en lugar de descubrirlos en tiempo de ejecución.

Abstracción de datos: Las clases genéricas permiten crear estructuras de datos y algoritmos que no están acoplados a un tipo de dato específico, lo que proporciona una mayor abstracción y flexibilidad en el diseño del código.

REFERENCIAS

- <https://www.w3schools.com/java/>
- <https://www.eclipse.org/downloads/packages/release/2022-03/r/eclipse-ide-enterprise-java-and-web-developers>
- <https://docs.oracle.com/javase/7/docs/api/java/util/List.html>
- <https://docs.oracle.com/javase/tutorial/java/generics/types.html>